

From Storage to Intelligence: The Evolution of Agent Memory Management

Botao Xie

225045044

Research Report for CSC6032 Course

School of Data Science
The Chinese University of Hong Kong, Shenzhen
April 2026

Abstract

Large language model (LLM) agents have demonstrated remarkable capabilities in complex reasoning and long-horizon planning. However, their stateless nature and limited context windows fundamentally constrain their effectiveness in sustained, long-term interactions. Agent memory systems have emerged as a critical solution, providing LLMs with the ability to store, manage, and evolve information across sessions.

This report surveys the evolution of agent memory management through three landmark systems. First, **MemGPT** [10] draws inspiration from operating system virtual memory to introduce hierarchical context management, enabling LLMs to handle unbounded context via intelligent swapping between in-context and external storage. Second, **A-MEM** [16] adopts the Zettelkasten methodology to construct a dynamic, self-evolving knowledge network, where memories are organized as interconnected notes that autonomously generate links and refine their representations over time. Third, **GAM** [14] proposes a hierarchical graph-based architecture that decouples rapid episodic buffering from stable semantic consolidation, effectively resolving the tension between context plasticity and long-term knowledge retention.

Together, these systems chart a clear progression in structural complexity: from one-dimensional linear storage, to two-dimensional planar graph structures, and finally to three-dimensional hierarchical multi-layer memory. This evolution reflects the broader trajectory of agent memory research—moving from simple data retention toward intelligent, structured knowledge management that supports robust long-term reasoning.

Contents

Abstract	i
1 Introduction	1
1.1 What is Agent Memory?	1
1.2 Why Agent Memory is Necessary	2
1.3 How Agent Memory Works	3
1.4 Report Structure	3
2 Background and Motivation	4
2.1 Challenges of Agent Memory	4
2.2 Structural Token-level Memory: A Taxonomy	5
3 MemGPT: Virtual Context Management	7
3.1 Motivation and Key Observation	7
3.2 System Architecture	8
3.3 Queue Manager and Function Executor	9
3.4 Performance Evaluation	10
3.4.1 Multi-Session Conversational Agents	10
3.4.2 Document Analysis	11
3.5 Summary	12
4 A-MEM: Agentic Memory with Zettelkasten	13
4.1 Motivation and Key Observation	13

4.2	System Architecture	14
4.3	Note Construction	14
4.4	Link Generation	15
4.5	Memory Evolution	15
4.6	Memory Retrieval	16
4.7	Performance Evaluation	16
4.7.1	LoCoMo Results	16
4.7.2	Memory Structure Visualization	17
4.8	Summary	17
5	GAM: Hierarchical Graph-based Memory	18
5.1	Motivation and Key Observation	18
5.2	System Architecture	19
5.3	Hierarchical Graph Memory Architecture	19
5.4	State-Based Memory Consolidation	20
5.4.1	Semantic Boundary Detection	20
5.4.2	Episodic Buffering State	21
5.4.3	Semantic Consolidation State	21
5.5	Graph-Guided Multi-Factor Retrieval	21
5.6	Performance Evaluation	22
5.6.1	LoCoMo Results	22
5.6.2	Partitioning Strategy Analysis	23
5.6.3	Efficiency Comparison	23
5.7	Summary	24
6	Conclusion and Future Work	26
6.1	Summary of the Survey	26
6.2	Future Work: Toward Multimodal Agent Memory	27
	References	29

List of Figures

1.1	Traditional memory system. The memory module acts as an intermediary between the LLM agent and its environment, supporting structured read and write operations.	1
2.1	Taxonomy of token-level memory organized by topological complexity. Memory architectures progress from flat 1D storage to planar 2D graphs and hierarchical 3D multi-layer structures [4].	5
3.1	MemGPT architecture. The LLM’s finite context window constitutes the main context, organized into system instructions, working context, and a FIFO queue. External context consists of archival and recall storage. The queue manager and function executor orchestrate data movement between tiers [10].	8
3.2	Document QA task performance vs. number of retrieved documents. Fixed-context baselines (GPT-4, GPT-3.5) degrade as more documents are added due to context overflow. MemGPT performance remains stable because it can iteratively page through archival storage [10].	11
4.1	A-MEM architecture. New interactions are converted into rich Zettelkasten-style notes (note construction), linked to related historical memories (link generation), and used to evolve neighboring notes (memory evolution) [16].	14
5.1	GAM architecture. State-based consolidation (left) switches between buffering and integration based on semantic topic boundaries. Hierarchical graph memory (center) separates global Topic Associative Network from local Event Progression Graphs. Graph-guided retrieval (right) performs top-down traversal with multi-factor ranking [14].	19

5.2	Comparison of partitioning strategies on LoCoMo. The semantic-boundary approach (GAM) outperforms heuristic alternatives across all metrics, with the largest margin in temporal reasoning [14]	24
-----	---	----

List of Tables

- 1.1 Context window lengths of commonly used LLMs (data circa 2024). The asterisk (*) denotes approximate message counts assuming a 1k-token system prompt and 50-token average message length. 2

- 3.1 Deep Memory Retrieval (DMR) performance on the MSC dataset. MemGPT significantly outperforms fixed-context baselines across all underlying models [10]. 10

- 4.1 Experimental results on the LoCoMo dataset (F1 scores, %). A-MEM achieves the best average ranking with significantly lower token consumption [16]. . . 17

- 5.1 GAM performance on LoCoMo (F1 scores, %). GAM achieves best average performance in most settings, particularly excelling at temporal reasoning [14]. 23
- 5.2 Efficiency comparison on LoCoMo. GAM consumes significantly fewer tokens per query while achieving better F1 scores, demonstrating that structured hierarchical organization is more cost-effective than flat or 2D alternatives [14]. 23

- 6.1 Comparative summary of the three surveyed agent memory systems. 27

Chapter 1

Introduction

1.1 What is Agent Memory?

Artificial intelligence agents based on large language models (LLMs) are increasingly deployed in settings that require sustained, multi-session interactions with users and dynamic environments. Tasks such as personalized assistance, autonomous research, and complex planning [11] demand that agents maintain coherent knowledge across exchanges that may span hours, days, or even weeks. At the core of this capability lies *agent memory*: the fundamental mechanism that empowers foundation models to store, manage, and evolve their experiences during long-term interactions [4, 10, 16].

Architecturally, an agent memory system is best understood as a *wrapper system* built upon an LLM. As illustrated in Figure 1.1, the memory system sits between the LLM agent and its surrounding environment, mediating the flow of information through structured read and write operations. The agent reads from memory to retrieve relevant historical context before generating a response, and writes to memory to record new experiences after each interaction. This layered design cleanly separates the agent’s reasoning engine from its knowledge management infrastructure, enabling the two concerns to evolve independently.

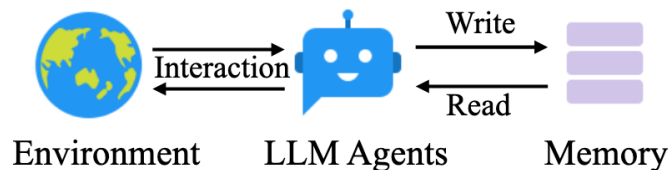


Figure 1.1: Traditional memory system. The memory module acts as an intermediary between the LLM agent and its environment, supporting structured read and write operations.

1.2 Why Agent Memory is Necessary

Three fundamental limitations of modern LLMs motivate the need for dedicated memory management systems.

Statelessness. Large language models are inherently stateless: each inference call processes only what is contained in the current input prompt, with no persistent internal state carried over from previous calls. Without external memory infrastructure, an LLM agent starts every session from scratch, unable to recall facts learned, preferences expressed, or tasks completed in prior interactions.

Limited context windows. Every LLM operates within a fixed-length context window that bounds the total amount of text—measured in tokens—that it can consider at once. As shown in Table 1.1, context lengths vary widely across models, ranging from a few thousand tokens in early open-source models to 128k tokens in recent commercial APIs. While this represents significant progress, many real-world tasks easily exceed these limits. A legal document review, a months-long user dialogue, or a comprehensive research report can surpass even the largest available context windows. Moreover, extending context through longer-sequence training incurs a quadratic increase in computational cost due to the self-attention mechanism [12], and recent research has shown that long-context models often fail to effectively utilize information placed in the middle of very long inputs.

Table 1.1: Context window lengths of commonly used LLMs (data circa 2024). The asterisk (*) denotes approximate message counts assuming a 1k-token system prompt and 50-token average message length.

Model / API	Context (tokens)	Approx. messages*
Llama 2	4k	~60
GPT-3.5 Turbo	16k	~300
GPT-4	32k	~600
Claude 2	100k	~2000
GPT-4 Turbo	128k	~2600

Cost and efficiency. Even when context windows are large enough to contain all relevant history, including the full conversation transcript in every inference call is expensive. Token cost grows linearly with context length, and latency increases quadratically for attention-based architectures. Intelligent memory management—selectively loading only the most relevant historical context—is therefore a practical necessity for cost-efficient long-term agent deployment [16, 14].

1.3 How Agent Memory Works

The lifecycle of agent memory can be decomposed into three stages, each addressing a distinct aspect of knowledge management [4]:

Formation. When an agent interacts with its environment, raw experiences must be encoded and stored in structured form. This step goes beyond simple logging: effective memory formation extracts semantically meaningful representations—keywords, contextual summaries, and relational metadata—that support later retrieval and reasoning. The quality of formation determines what information is available to the agent in future sessions.

Evolution. Long-term memory is not static. As new experiences accumulate, existing memories may become outdated, contradicted, or enriched by later information. A robust memory system must therefore support *dynamic evolution*: the process of reflecting on new input and updating existing memory representations to maintain factual consistency and conceptual coherence. This mimics the human cognitive process of memory consolidation, in which short-term experiences are integrated into long-term knowledge structures.

Retrieval. At inference time, the agent must efficiently identify and surface the subset of stored memories most relevant to the current query. Retrieval mechanisms range from simple keyword matching and recency-based ranking to sophisticated semantic search over dense vector embeddings. The precision of retrieval directly determines the quality of the context provided to the LLM, and thus the accuracy of its generated responses.

1.4 Report Structure

This report surveys three systems that represent landmark advances along the trajectory of agent memory research. Chapter 2 establishes the technical challenges motivating this work and introduces a taxonomic framework for classifying memory architectures by their structural complexity. Chapters 3, 4, and 5 present detailed analyses of MemGPT, A-MEM, and GAM, respectively, covering their motivations, designs, and experimental evaluations. Chapter 6 synthesizes the key insights and discusses the direction of future work.

Chapter 2

Background and Motivation

2.1 Challenges of Agent Memory

Managing agent memory is a complex system-engineering task that extends far beyond simple data storage. Three primary technical challenges define the current frontier of agent memory research [4]:

Accuracy. An agent’s memory must support precise retrieval and maintain factual consistency across sessions. Two failure modes are particularly problematic. First, *stale data*: information stored in earlier sessions may be superseded by later interactions, and a system that retrieves outdated facts can cause the agent to produce incorrect or contradictory responses. Second, *misinterpretation*: imprecise memory encoding can distort the meaning of an experience, leading to retrieval results that superficially match a query but are semantically incorrect. Both failure modes manifest as hallucinations in the agent’s output—confident statements that are inconsistent with the actual interaction history.

Efficiency. Memory operations are performed at inference time, directly in the critical path of each agent response. Frequent read and write operations over large memory repositories can introduce significant computational latency and consume substantial token budgets. As agents are increasingly deployed in latency-sensitive applications and billed per token, the efficiency of memory management has direct impact on both user experience and operational cost [14].

Scalability. Over the lifetime of a deployed agent, the memory repository grows continuously as new interactions are logged. A system that performs adequately with hundreds of stored memories may degrade severely at the scale of thousands or tens of thousands of entries. Scalable memory systems must maintain retrieval quality and response

latency as the repository size increases, ideally with sub-linear growth in computational cost.

2.2 Structural Token-level Memory: A Taxonomy

In response to these challenges, the research community has developed a family of *structural token-level memory* architectures—systems that organize LLM memory as explicit, structured repositories of text tokens rather than relying solely on the model’s implicit parametric knowledge [4]. These architectures can be categorized along a single dimension of *topological complexity*, yielding a three-level taxonomy illustrated in Figure 2.1:

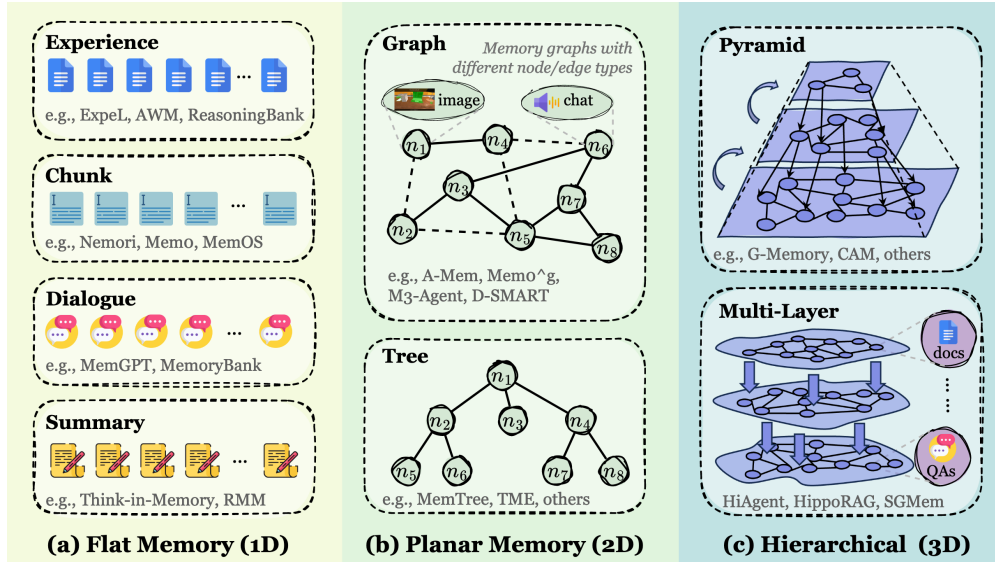


Figure 2.1: Taxonomy of token-level memory organized by topological complexity. Memory architectures progress from flat 1D storage to planar 2D graphs and hierarchical 3D multi-layer structures [4].

Flat (1D) Memory. The simplest category treats the memory repository as a linear sequence of discrete units—individual dialogue turns, experience chunks, or text passages stored as a flat list. Early memory systems, such as basic dialogue logs and dense-retrieval interaction stores [7], fall into this category. Flat memory is computationally straightforward and easy to implement, but it provides no structural information about the relationships between stored items. Retrieval must rely entirely on content similarity, and the absence of relational structure makes it difficult to resolve conflicts, detect redundancies, or support multi-hop reasoning over stored knowledge. MemGPT [10], discussed in Chapter 3, is a representative system that organizes memory as a hierarchy of flat text chunks, distinguished primarily by their storage tier rather than their relational structure.

Planar (2D) Memory. More sophisticated architectures organize memories as nodes in a graph or tree, where edges capture semantic, temporal, or causal relationships between memory units [2, 3]. This planar structure provides the retrieval mechanism with relational context: rather than treating each memory in isolation, the system can follow edges to discover associated knowledge and reason over clusters of related information. Graph-based memory also supports more nuanced operations, such as propagating updates along semantic links when a stored fact is revised. A-MEM [16], presented in Chapter 4, exemplifies 2D memory: it organizes memories as an interconnected network of Zettelkasten-style notes, where links are established dynamically based on semantic similarity and LLM-inferred relationships.

Hierarchical (3D) Memory. The most structurally sophisticated category extends the planar model to multiple layers, each operating at a different level of semantic granularity. Cross-layer links connect fine-grained episodic details to high-level abstract themes, enabling the agent to navigate fluidly between granular evidence and conceptual knowledge. This hierarchical organization is particularly well-suited to long-horizon interactions, where the agent must simultaneously track the immediate context of an ongoing conversation and maintain stable knowledge of long-term user preferences or domain facts. GAM [14], analyzed in Chapter 5, instantiates 3D memory through a dual-layer graph architecture that separates a global topic network from local event progression graphs, with cross-layer links bridging the two.

This three-level taxonomy provides the organizing framework for the remainder of this report. The three systems surveyed—MemGPT, A-MEM, and GAM—represent successive steps along this axis of topological complexity, each advancing the state of the art by introducing richer structure into the memory substrate.

Chapter 3

MemGPT: Virtual Context Management

3.1 Motivation and Key Observation

The central challenge addressed by MemGPT [10] is the finite context window of large language models. When a conversation grows beyond the model’s maximum input length, older messages must be discarded or compressed, causing the agent to lose access to potentially critical long-term information. Standard mitigations—such as recursive summarization or truncation—degrade the quality of the available context and introduce irreversible information loss.

MemGPT’s foundational insight is a structural analogy between LLM context windows and main memory in a traditional operating system. In classical OS design, physical RAM is a scarce, finite resource. Applications running on modern systems require far more memory than physically exists, and the OS resolves this tension through *virtual memory paging*: data that does not fit in RAM is transparently moved to disk storage, and retrieved on demand when the application accesses it. From the application’s perspective, the system appears to have unlimited memory.

MemGPT applies this same principle to LLM agents. The fixed-length context window plays the role of physical RAM: it is the “active working memory” that the LLM processor can directly access during inference. External storage systems—a vector database and a message archive—play the role of disk: they hold vast amounts of information that cannot fit into the context window simultaneously, but can be explicitly retrieved when needed. By equipping the LLM with function-calling capabilities to manage its own memory, MemGPT creates the illusion of an unbounded context window while operating within the strict token

limits of the underlying model.

3.2 System Architecture

MemGPT organizes its memory into two primary tiers: *main context* (analogous to RAM) and *external context* (analogous to disk), as illustrated in Figure 3.1.

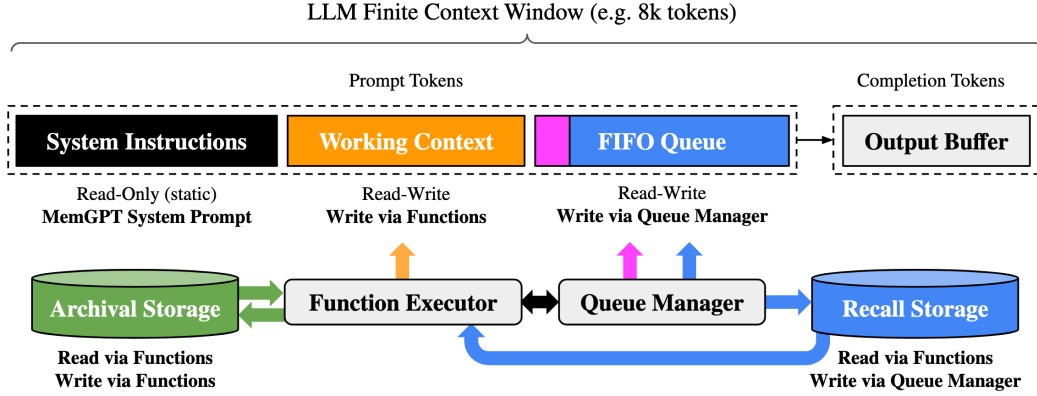


Figure 3.1: *MemGPT architecture. The LLM’s finite context window constitutes the main context, organized into system instructions, working context, and a FIFO queue. External context consists of archival and recall storage. The queue manager and function executor orchestrate data movement between tiers [10].*

Main context (prompt tokens) is the in-context memory directly accessible to the LLM during inference. It is subdivided into three contiguous segments:

- **System instructions:** A read-only block containing the MemGPT control flow specification, a description of the memory hierarchy, the policies governing memory management decisions, and the schema of available function calls. This section is fixed at system initialization and never modified at runtime.
- **Working context:** A fixed-size read/write block of unstructured text, writable only through MemGPT function calls. In conversational settings, working context stores core persistent facts: key user preferences, biographical details, the agent’s adopted persona, and any information that should be immediately accessible without a retrieval query. The agent autonomously decides what to store here based on its assessment of long-term importance.
- **FIFO queue:** A rolling buffer that stores the recent history of messages, including user turns, agent responses, system alerts (e.g., memory pressure warnings), and the inputs and outputs of function calls. The queue operates on a first-in, first-out eviction policy.

When the queue is full, the oldest messages are evicted and a recursive summary of the evicted content is prepended to the queue, preserving a compressed record of the lost context.

External context encompasses all information held outside the context window:

- **Recall storage:** A searchable archive of the full message history, including all messages ever evicted from the FIFO queue. The queue manager writes both incoming user messages and generated agent responses to recall storage in real time. Information in recall storage can be retrieved on demand via MemGPT function calls and inserted into the active context window.
- **Archival storage:** A read/write database for storing arbitrary-length text objects—documents, notes, or structured data—that the agent wishes to retain for long-term reference. In document analysis tasks, for example, the full content of a lengthy document can be preloaded into archival storage, and the agent queries it iteratively to extract relevant passages.

3.3 Queue Manager and Function Executor

Two components orchestrate the data movement between memory tiers:

Queue Manager. The queue manager monitors the token count of the prompt and enforces the eviction policy. When the total prompt length approaches a “warning threshold” (e.g., 70% of the context limit), the queue manager inserts a system alert into the FIFO queue notifying the LLM of impending memory pressure. This gives the LLM processor an opportunity to proactively save important information—for example, by writing key facts from the queue into working context or archival storage—before eviction occurs. When the prompt exceeds the “flush threshold” (e.g., 100% of the context limit), the queue manager forcibly evicts a batch of messages, generates a new recursive summary incorporating the evicted content, and prepends it to the queue. Evicted messages are permanently removed from main context but remain indefinitely accessible in recall storage.

Function Executor. MemGPT exposes a set of function calls that the LLM can invoke to manipulate its own memory. These include operations for reading and writing to working context, searching recall storage by keyword or semantic similarity, and reading from or writing to archival storage. When the LLM generates an output that encodes a function call, the function executor parses and validates the call, executes it, and feeds the result back into the LLM’s context as a system message. This creates a feedback loop in which the

LLM can observe the outcomes of its memory operations and adjust its subsequent behavior accordingly.

Function chaining is a key capability that enables MemGPT to perform multi-step retrieval. By attaching a special “request heartbeat” flag to a function call output, the LLM can request an immediate follow-up inference cycle rather than waiting for the next user message. This allows the agent to iteratively page through search results, navigate nested data structures, or progressively refine a query across multiple function calls—all within a single user interaction.

3.4 Performance Evaluation

MemGPT is evaluated on two task families where fixed-context LLMs are fundamentally limited: multi-session conversational agents and long-document analysis.

3.4.1 Multi-Session Conversational Agents

The authors evaluate MemGPT on the Multi-Session Chat (MSC) dataset [15], assessing two criteria: *consistency* (the ability to recall facts from prior sessions) and *engagement* (the ability to craft personalized dialogue drawing on long-term user knowledge).

Deep Memory Retrieval (DMR). In the DMR task, the agent is asked a question that can only be answered using information discussed in a prior session (sessions 1–5). The fixed-context baselines receive a lossy recursive summary of past sessions, while MemGPT has access to the full interaction history via paginated recall search. Results are shown in Table 3.1.

Table 3.1: *Deep Memory Retrieval (DMR) performance on the MSC dataset. MemGPT significantly outperforms fixed-context baselines across all underlying models [10].*

Model	Accuracy (%)	ROUGE-L (R)
GPT-3.5 Turbo	38.7	0.394
+ MemGPT	66.9	0.629
GPT-4	32.1	0.296
+ MemGPT	92.5	0.814
GPT-4 Turbo	35.3	0.359
+ MemGPT	93.4	0.827

The results reveal a striking pattern: without MemGPT, all three models achieve accuracy in the range of 32–39%, reflecting the severe bottleneck imposed by the finite context window.

With MemGPT, performance jumps dramatically—GPT-4 Turbo reaches 93.4% accuracy. This confirms that the bottleneck is not the model’s reasoning capability but its inability to access long-term information without structured memory management.

3.4.2 Document Analysis

In the document QA task, the agent is presented with a set of Wikipedia documents and asked a question from the NaturalQuestions-Open dataset. The number of retrieved documents K is varied to test how performance scales with context length.

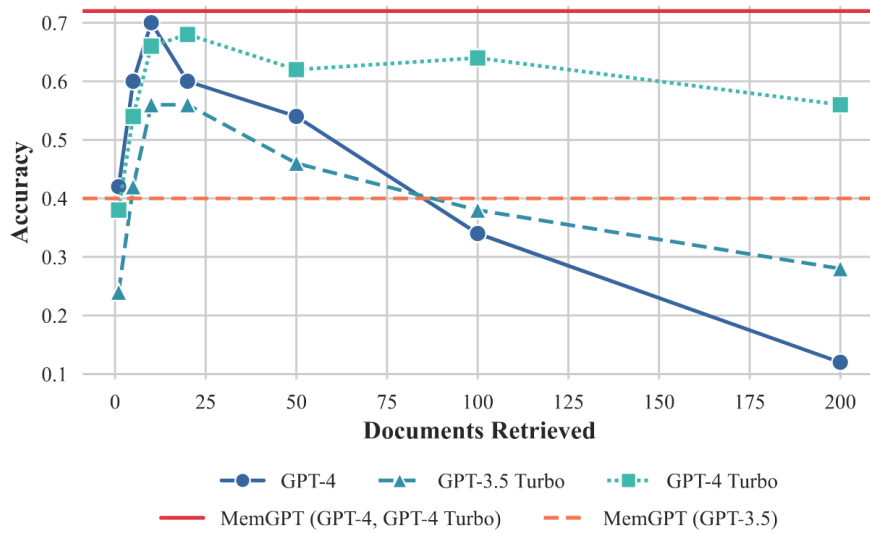


Figure 3.2: Document QA task performance vs. number of retrieved documents. Fixed-context baselines (GPT-4, GPT-3.5) degrade as more documents are added due to context overflow. MemGPT performance remains stable because it can iteratively page through archival storage [10].

As shown in Figure 3.2, fixed-context baselines exhibit a characteristic pattern: accuracy initially improves as more relevant documents are added, then degrades as the total context exceeds the model’s window and truncation removes critical information. MemGPT, by contrast, maintains stable accuracy regardless of the number of documents, because it can iteratively query archival storage rather than loading all documents simultaneously into the context window. This decoupling of performance from the physical context window size is the key practical advantage of MemGPT’s OS-inspired design.

3.5 Summary

MemGPT establishes the feasibility of virtual context management for LLM agents. By treating the context window as a constrained memory resource and equipping the LLM with OS-inspired function calls to manage its own memory hierarchy, MemGPT enables persistent, long-term conversational agents and large-scale document analysis tasks that far exceed what fixed-context models can handle. The system represents the first step in the structural evolution of agent memory: a 1D architecture in which memories are organized as discrete text chunks arranged in a linear hierarchy of storage tiers, distinguished by access speed and capacity rather than relational structure.

Chapter 4

A-MEM: Agentic Memory with Zettelkasten

4.1 Motivation and Key Observation

While MemGPT addresses context *capacity*, A-MEM [16] targets memory *quality*. Three limitations of prior systems motivate this work.

Rigid schemas. Existing systems require predefined storage structures and fixed access patterns, limiting adaptability across diverse tasks. When an agent encounters a novel situation, its memory schema becomes a constraint rather than a scaffold.

Manual management. Most systems require developers to explicitly specify when to store information and what retrieval triggers to use. This fundamentally limits agent autonomy: the agent can only remember what it was programmed to remember.

Static knowledge. Prior systems treat stored memories as immutable records. As new interactions accumulate, earlier memories should be enriched, corrected, or linked to later discoveries—but static systems fail to support this evolution.

The key insight of A-MEM is that all three limitations can be addressed by adopting the *Zettelkasten* method as the design principle for agent memory. Developed by sociologist Niklas Luhmann [8], Zettelkasten organizes information as atomic, self-contained notes interconnected through dynamic links, forming a decentralized knowledge graph that grows organically without predefined categories.

4.2 System Architecture

A-MEM implements three coordinated modules—note construction, link generation, and memory evolution—that collectively realize the Zettelkasten principle. The architecture is illustrated in Figure 4.1.

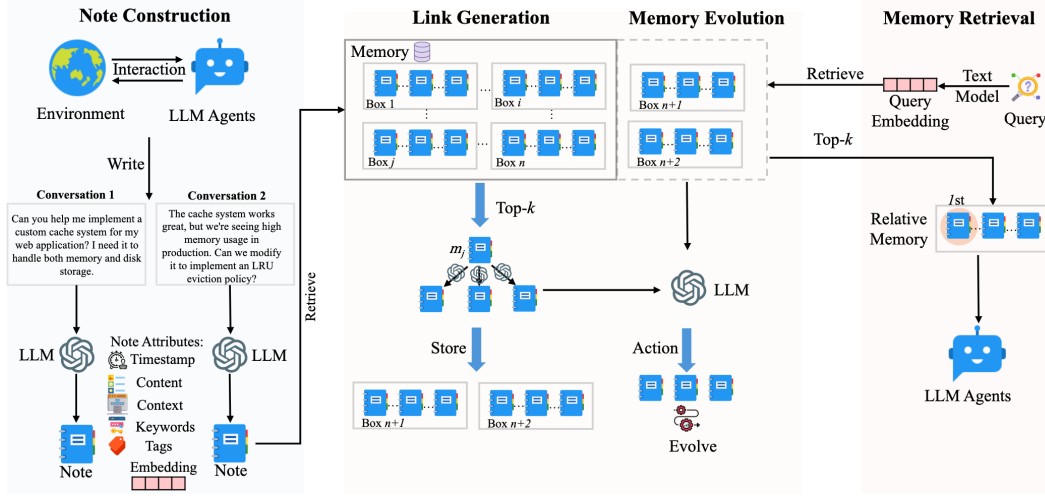


Figure 4.1: A-MEM architecture. New interactions are converted into rich Zettelkasten-style notes (note construction), linked to related historical memories (link generation), and used to evolve neighboring notes (memory evolution) [16].

4.3 Note Construction

When an agent interacts with its environment, the raw interaction is processed by an LLM to construct a rich, multi-attribute memory note. Each note m_i in the collection $\mathcal{M} = \{m_1, \dots, m_N\}$ is represented as:

$$m_i = \{c_i, t_i, K_i, G_i, X_i, e_i, L_i\}, \quad (4.1)$$

where c_i is the original content, t_i is the timestamp, K_i is a set of LLM-generated keywords, G_i contains LLM-generated tags, X_i is an LLM-generated contextual description, e_i is a dense embedding vector, and L_i is the set of links to related notes.

The semantic components are generated by prompting the LLM with template P_{s1} :

$$K_i, G_i, X_i \leftarrow \text{LLM}(c_i \parallel t_i \parallel P_{s1}). \quad (4.2)$$

The embedding is computed by encoder f_{enc} over all textual components:

$$e_i = f_{\text{enc}}[\text{concat}(c_i, K_i, G_i, X_i)]. \quad (4.3)$$

This multi-faceted structure serves two purposes: the LLM-generated components capture implicit semantic knowledge beyond the raw content, while the dense embedding enables efficient similarity-based retrieval.

4.4 Link Generation

After a new note m_n is constructed, A-MEM autonomously establishes connections to existing memories in two stages.

Embedding-based candidate retrieval. Cosine similarity between m_n and every existing note m_j is computed:

$$s_{n,j} = \frac{e_n \cdot e_j}{|e_n| |e_j|}. \quad (4.4)$$

The top- k most similar notes form the candidate set:

$$\mathcal{M}_{\text{near}}^n = \{m_j \mid \text{rank}(s_{n,j}) \leq k, m_j \in \mathcal{M}\}. \quad (4.5)$$

LLM-driven link establishment. The candidates are passed to the LLM with prompt P_{s2} :

$$L_i \leftarrow \text{LLM}(m_n \parallel \mathcal{M}_{\text{near}}^n \parallel P_{s2}). \quad (4.6)$$

The LLM can detect causal relationships, thematic commonalities, and subtle conceptual connections that embedding similarity alone cannot capture. Linked notes are organized into *semantic boxes*: clusters of mutually connected notes sharing contextual descriptions—the digital analogue of thematic filing boxes in a physical Zettelkasten. A single note can belong to multiple boxes simultaneously, reflecting the multi-dimensional nature of real-world knowledge.

4.5 Memory Evolution

After linking, A-MEM evolves existing memories in response to the new note. For each neighbor $m_j \in \mathcal{M}_{\text{near}}^n$, an evolved note m_j^* is generated:

$$m_j^* \leftarrow \text{LLM}(m_n \parallel \mathcal{M}_{\text{near}}^n \setminus \{m_j\} \parallel m_j \parallel P_{s3}). \quad (4.7)$$

The evolved note replaces m_j in the collection $\mathcal{M}$. The LLM may update any combination of the note’s contextual description, keywords, tags, and links. This evolutionary mechanism produces a compounding effect: as the memory collection grows, each new note triggers updates to its neighbors, which may have themselves been updated by prior interactions. The result is a knowledge network that becomes progressively richer and more interconnected over time—mirroring the human process of deepening understanding through continuous recontextualization.

4.6 Memory Retrieval

During inference, the query q is embedded as $e_q = f_{\text{enc}}(q)$, and the top- k most similar notes are retrieved by cosine similarity:

$$\mathcal{M}_{\text{retrieved}} = \left\{ m_i \mid \text{rank} \left(\frac{e_q \cdot e_i}{|e_q||e_i|} \right) \leq k, m_i \in \mathcal{M} \right\}. \quad (4.8)$$

Because notes within a semantic box are linked, retrieving any note in a box automatically surfaces its connected neighbors, providing the agent with a cluster of contextually related information rather than a single isolated record.

4.7 Performance Evaluation

A-MEM is evaluated on the LoCoMo dataset [9], a long-term conversational benchmark with dialogues averaging 9k tokens across up to 35 sessions and five question categories: multi-hop, temporal, open-domain, single-hop, and adversarial. F1 score and BLEU-1 are used as primary metrics.

4.7.1 LoCoMo Results

Table 4.1 summarizes results for two representative models. Baselines include LoCoMo [9], ReadAgent [6], MemoryBank [17], and MemGPT [10]. A-MEM consistently achieves the best average ranking across categories, with particularly strong performance in the Temporal category—reflecting the effectiveness of memory evolution in maintaining temporal coherence. Crucially, A-MEM achieves these results while consuming far fewer tokens than competing systems (2,520 vs. 16,977 for MemGPT with GPT-4o-mini), delivering a compelling “better-for-less” trade-off.

Table 4.1: Experimental results on the LoCoMo dataset (F1 scores, %). A-MEM achieves the best average ranking with significantly lower token consumption [16].

Model	Method	Multi-Hop	Temporal	Open-Dom.	Single-Hop	Avg Rank	Tokens
GPT-4o-mini	LoCoMo	25.02	18.41	12.04	40.36	2.4	16,910
	ReadAgent	9.15	12.60	5.31	9.67	4.2	643
	MemoryBank	5.00	9.68	5.56	6.61	4.8	432
	MemGPT	26.65	25.52	9.15	41.04	2.4	16,977
	A-MEM	27.02	45.85	12.14	44.65	1.2	2,520
GPT-4o	LoCoMo	28.00	9.09	16.47	61.56	2.0	16,910
	ReadAgent	14.61	4.16	8.84	12.46	4.0	805
	MemoryBank	6.49	2.47	6.43	8.28	5.0	569
	MemGPT	30.36	17.29	12.24	60.16	2.4	16,987
	A-MEM	32.86	39.41	17.10	48.43	1.6	1,216

4.7.2 Memory Structure Visualization

T-SNE dimensionality reduction applied to stored note embeddings provides a qualitative confirmation of A-MEM’s organizing capability. Base memory embeddings are scattered uniformly, reflecting the absence of any imposed structure. A-MEM embeddings form tight, distinct clusters: related topics are co-located in the embedding space, demonstrating that the Zettelkasten-inspired architecture successfully imposes semantically meaningful organization.

4.8 Summary

A-MEM advances agent memory from 1D linear storage to a 2D planar graph, where nodes are rich Zettelkasten-style notes and edges are autonomously generated semantic links. Three properties distinguish A-MEM from prior work: (1) LLM-driven note construction captures implicit semantic content beyond raw text; (2) embedding-filtered, LLM-driven link generation establishes content-aware connections without predefined schemas; and (3) memory evolution continuously refines the knowledge network as new experiences accumulate. The result is the first agent memory system that exhibits genuine autonomy in its own organization—adapting its structure dynamically to the content and context of the interactions it processes.

Chapter 5

GAM: Hierarchical Graph-based Memory

5.1 Motivation and Key Observation

Despite the structural advances of A-MEM, systems that maintain a continuous, evolving stream of memory face two persistent problems in very long-horizon interactions [14, 13].

Semantic drift. When distinct topics are handled by the same continuous memory stream, their representations can become conflated. For example, a memory system that mixes a discussion about Apple (the technology company) with a later discussion about apple (the fruit) may incorrectly merge these topics, distorting the agent’s thematic consistency. Semantic drift occurs when continuous updates cause topic boundaries to blur, undermining the reliability of long-term knowledge.

Memory loss. In large memory graphs, established nodes can become structurally isolated as new information floods in. Nodes with few new connections are progressively pushed to the periphery of the graph, weakened in terms of retrieval probability, and eventually effectively forgotten—even though the information they encode may still be relevant.

The key observation motivating GAM is that human memory naturally groups experiences by topics rather than maintaining a monolithic continuous stream. The brain processes new information in an episodic buffer before selectively consolidating it into long-term semantic memory—and crucially, this consolidation happens only at the boundaries between distinct episodes, not continuously during an ongoing experience.

GAM operationalizes this insight through two core mechanisms: a *Hierarchical Graph*

Memory Architecture that physically separates global semantic knowledge from local episodic details, and a *State-Based Memory Consolidation* mechanism that triggers integration only when a semantic topic boundary is detected. Together, these mechanisms decouple rapid context perception from stable knowledge retention.

5.2 System Architecture

The GAM framework operates through three coordinated stages, as illustrated in Figure 5.1: episodic buffering, semantic consolidation, and graph-guided retrieval.

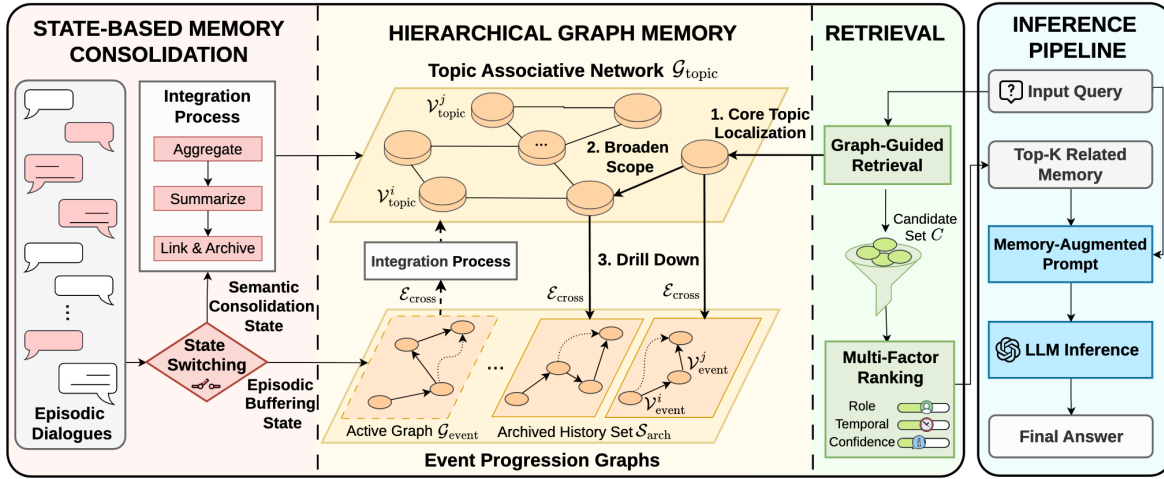


Figure 5.1: GAM architecture. State-based consolidation (left) switches between buffering and integration based on semantic topic boundaries. Hierarchical graph memory (center) separates global Topic Associative Network from local Event Progression Graphs. Graph-guided retrieval (right) performs top-down traversal with multi-factor ranking [14].

5.3 Hierarchical Graph Memory Architecture

GAM defines a time-varying memory structure $\mathcal{H}_t$ as a composite of four components:

$$\mathcal{H}_t = \{G_{\text{topic}}^{(t)}, G_{\text{event}}^{(t)}, S_{\text{arch}}^{(t)}, E_{\text{cross}}^{(t)}\}, \quad (5.1)$$

where $G_{\text{topic}}^{(t)}$ is the global Topic Associative Network, $G_{\text{event}}^{(t)}$ is the active Event Progression Graph, $S_{\text{arch}}^{(t)}$ is the archive of past event graphs, and $E_{\text{cross}}^{(t)}$ are cross-layer edges linking topics to archived episodes.

Topic Associative Network (G_{topic}) serves as the long-term global memory. Its nodes

V_{topic} represent high-level semantic clusters and abstract themes derived from consolidated historical interactions. Its edges E_{topic} capture deep semantic correlations between themes, weighted by LLM-computed similarity scores. This layer is the stable, slowly-evolving backbone of the agent’s world model.

Event Progression Graph (G_{event}) is the active, rapidly-updated local buffer. Its nodes V_{event} are atomic interaction units—individual utterances or system responses—captured in real time. Its edges E_{event} model the temporal and causal flow of the current dialogue. This layer is optimized for fast append operations, minimizing encoding latency.

Cross-layer associations (E_{cross}) bridge the two layers: they connect topic nodes in G_{topic} to the archived snapshots of the event graphs (stored in S_{arch}) that gave rise to each topic. These links enable the retrieval mechanism to navigate from high-level themes to the precise episodic evidence that supports them.

The two-layer design is illustrated in Figure 5.1.

5.4 State-Based Memory Consolidation

GAM models memory management as a two-state finite machine, illustrated in Figure 5.1. Transitions between states are triggered by semantic boundary detection—a key innovation that replaces the arbitrary time-window or token-count triggers used by prior systems.

5.4.1 Semantic Boundary Detection

The boundary condition is formalized as a semantic divergence indicator $b_t \in \{0, 1\}$:

$$b_t = \mathbb{I}(\Delta(G_{\text{event}}^{(t)}, G_{\text{topic}}^{(t)}) > \epsilon), \quad (5.2)$$

where $\Delta(\cdot)$ measures semantic divergence between the current episode and the global memory, and ϵ is a tolerance threshold. Rather than computing this distance explicitly—which would be computationally expensive—GAM uses an LLM as a neural proxy for the thresholding function. The LLM is queried at sparse maintenance events (session end, natural pauses, or buffer overflow) to determine whether the accumulated local context has diverged significantly enough from the global state to warrant consolidation.

5.4.2 Episodic Buffering State

In this state, the system focuses on capturing the current dialogue with minimal overhead. Each incoming utterance is constructed as an atomic event unit e_t and appended to the local graph:

$$V_{\text{event}}^{(t)} \leftarrow V_{\text{event}}^{(t-1)} \cup \{e_t\}, \quad E_{\text{event}}^{(t)} \leftarrow E_{\text{event}}^{(t-1)} \cup E_{\text{temp}}^{(t)}, \quad (5.3)$$

where $E_{\text{temp}}^{(t)}$ are sequential edges linking the new event to its immediate predecessors. Crucially, the global Topic Associative Network G_{topic} is not modified during this state, providing *write isolation*: transient, potentially noisy information is contained within the local buffer and cannot contaminate the global knowledge store.

5.4.3 Semantic Consolidation State

When $b_t = 1$, the system transitions to the consolidation state. The buffered event graph is transformed into a new topic node v_{new} with a dual-granularity representation:

$$v_{\text{new}} = \{c_{\text{sum}}, c_{\text{raw}}\}, \quad (5.4)$$

where c_{sum} is an LLM-generated summary capturing high-level semantic content (enabling abstract thematic reasoning), and c_{raw} is the concatenation of all raw event text (preserving fine-grained details for precise retrieval).

The new node is integrated into the global network through a coarse-to-fine candidate selection strategy: c_{sum} first retrieves the top-5 nearest existing topic nodes by vector similarity, and only this compact candidate set is passed to the LLM-based semantic scorer to evaluate relationship types and confidence scores. New edges E_{new} are established when confidence exceeds a threshold τ :

$$V_{\text{topic}}^{(t)} \leftarrow V_{\text{topic}}^{(t-1)} \cup \{v_{\text{new}}\}, \quad E_{\text{topic}}^{(t)} \leftarrow E_{\text{topic}}^{(t-1)} \cup E_{\text{new}}. \quad (5.5)$$

After integration, the event graph is archived in S_{arch} and linked to v_{new} via a cross-layer edge. The active local buffer is reset, and the system returns to the Episodic Buffering State.

5.5 Graph-Guided Multi-Factor Retrieval

At inference time, GAM employs a three-stage top-down traversal to retrieve precise, contextually grounded information from the hierarchical memory.

Stage 1 – Semantic Anchoring and Expansion. The query is used to identify the top- k

most relevant topic nodes in G_{topic} by vector similarity. The search scope is then expanded by including first-order neighbors of these anchor nodes:

$$V_{\text{anchor}} = V_{\text{top}} \cup \{v \in V_{\text{topic}} \mid \exists u \in V_{\text{top}}, (u, v) \in E_{\text{topic}}\}. \quad (5.6)$$

Stage 2 – Structural Drill-Down. From the anchor set, the system traverses cross-layer links E_{cross} to access the specific archived Event Progression Graphs associated with each anchor topic, aggregating their event nodes into a candidate set $\mathcal{C}$:

$$\mathcal{C} = \bigcup_{u \in V_{\text{anchor}}} \{v \in V(G') \mid G' \in S_{\text{arch}}, (u, G') \in E_{\text{cross}}\}. \quad (5.7)$$

Stage 3 – Multi-Factor Re-ranking. Candidates in $\mathcal{C}$ are re-ranked using a composite scoring function that combines semantic relevance with three contextual signals:

$$\text{Score}(v, q) = P_{\text{sem}}(v \mid q) \cdot \prod_{k \in \mathcal{K}} \beta_k^{\mathbb{I}_k(v, q)}, \quad (5.8)$$

where $\mathcal{K} = \{\text{time}, \text{conf}, \text{role}\}$. The temporal factor β_{time} boosts memories containing time expressions relevant to time-sensitive queries. The confidence factor β_{conf} prioritizes information that passed self-consistency verification during encoding. The role factor β_{role} disentangles multi-party narratives by verifying alignment between the memory source and the target speaker implied by the query.

5.6 Performance Evaluation

GAM is evaluated on two benchmarks: LoCoMo [9] (long open-domain dialogues) and LongDialQA (multi-party TV script dialogues). Baselines span three paradigms: heuristic approaches (MemoryBank [17], ReadAgent [6]), OS-inspired unified stream systems (MemGPT [10], MemoryOS [5], Mem0 [1]), and the self-evolving agent A-MEM [16].

5.6.1 LoCoMo Results

Table 5.1 presents results on LoCoMo. GAM achieves the best average F1 score in most backbone configurations. It is particularly strong in complex temporal reasoning tasks: with the Qwen-2.5-7B backbone, GAM surpasses Mem0 by over 18% in Temporal F1. Even on GPT-4o-mini, where commercial baselines are strong, GAM maintains the highest Average F1 of 43.14. These gains confirm that the dual-phase architecture provides a more robust structural foundation for long-term memory than purely retrieval-optimized systems.

Table 5.1: GAM performance on LoCoMo (F1 scores, %). GAM achieves best average performance in most settings, particularly excelling at temporal reasoning [14].

Model	Method	Multi-Hop	Temporal	Open-Dom.	Single-Hop	Avg F1
GPT-4o-mini	ReadAgent	9.15	12.60	5.31	9.67	9.18
	MemoryBank	5.00	9.68	5.56	6.61	6.71
	MemGPT	26.65	25.52	9.15	41.04	25.59
	A-MEM	27.02	<u>45.85</u>	12.14	44.65	32.41
	MemoryOS	<u>35.27</u>	41.15	<u>20.02</u>	<u>48.62</u>	<u>36.27</u>
	Mem0	32.78	56.42	20.45	51.81	40.37
	GAM	35.88	<u>51.96</u>	28.12	56.58	43.14
Qwen-2.5-7B	ReadAgent	5.37	2.82	5.86	5.23	4.82
	MemoryBank	6.22	2.58	8.53	6.17	5.88
	MemGPT	6.17	2.18	6.33	8.95	5.91
	A-MEM	21.85	27.54	14.16	33.25	24.20
	MemoryOS	<u>29.37</u>	<u>28.96</u>	<u>20.22</u>	<u>36.89</u>	<u>28.86</u>
	Mem0	32.86	41.22	18.24	49.21	35.38
	GAM	35.32	48.97	21.14	54.58	40.00

5.6.2 Partitioning Strategy Analysis

GAM evaluates different event graph partitioning strategies to validate the semantic-boundary approach. Figure 5.2 shows that semantic boundary detection consistently outperforms heuristic baselines (fixed window size, sentence count) across all evaluation categories, with the most significant advantage in temporal reasoning tasks.

5.6.3 Efficiency Comparison

Table 5.2 highlights the computational efficiency gains of GAM’s hierarchical structure relative to A-MEM and other baselines on LoCoMo.

Table 5.2: Efficiency comparison on LoCoMo. GAM consumes significantly fewer tokens per query while achieving better F1 scores, demonstrating that structured hierarchical organization is more cost-effective than flat or 2D alternatives [14].

Method	Tokens/Q	Time (s)	F1
A-MEM	4,221.12	2.21	24.20
MemoryOS	3400.41	154.22	28.86
Mem0	1533.94	0.51	35.38
GAM	1,370.18	0.80	40.00

GAM consumes approximately 1,370 tokens per query, compared to 4,221 for A-MEM—a

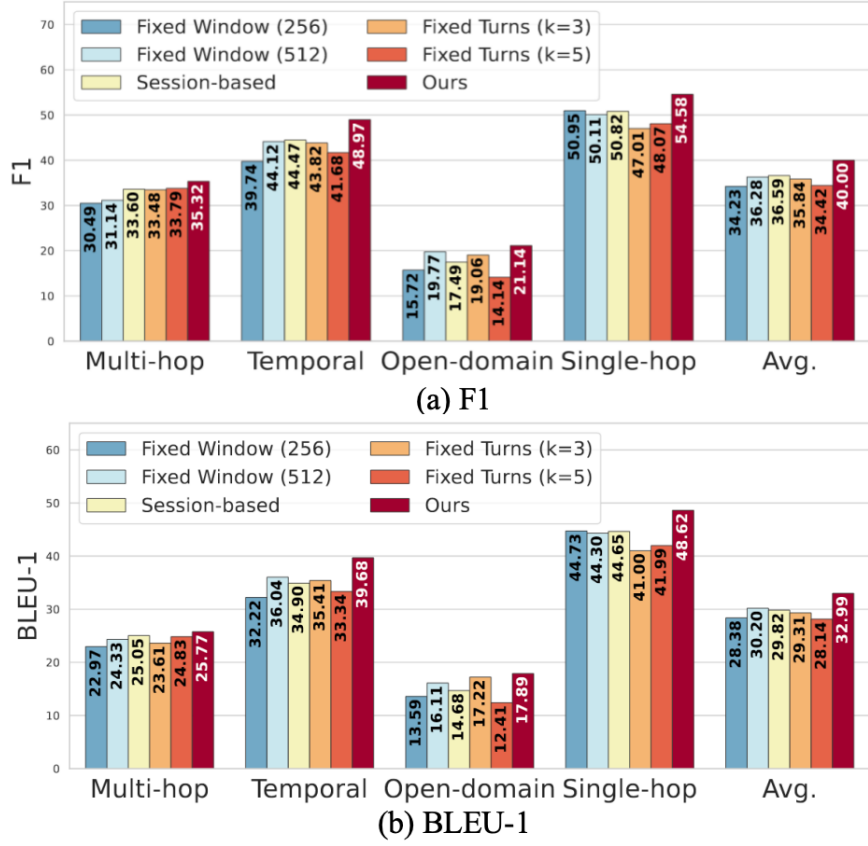


Figure 5.2: Comparison of partitioning strategies on LoCoMo. The semantic-boundary approach (GAM) outperforms heuristic alternatives across all metrics, with the largest margin in temporal reasoning [14].

67% reduction. This efficiency gain arises directly from the hierarchical architecture: the top-down retrieval traversal narrows the candidate set at each stage, avoiding the exhaustive cross-attention over the full memory graph required by 2D approaches.

5.7 Summary

GAM extends the structural evolution of agent memory to three dimensions by introducing a hierarchical, multi-layer graph architecture. The key innovations are: (1) physical separation of the global Topic Associative Network from local Event Progression Graphs, providing write isolation that prevents transient noise from contaminating stable knowledge; (2) LLM-driven semantic boundary detection that triggers consolidation at semantically complete episode boundaries rather than at arbitrary time or token thresholds; and (3) graph-guided multi-factor retrieval that combines top-down hierarchical traversal with temporal, confidence, and role-centric signal modulation. The result is a system that simulta-

neously achieves superior reasoning accuracy and computational efficiency—demonstrating that richer structural organization in memory design yields benefits at every level of the performance-efficiency trade-off.

Chapter 6

Conclusion and Future Work

6.1 Summary of the Survey

This report has traced the evolution of agent memory management through three landmark systems, each representing a step forward in the structural sophistication of memory organization.

MemGPT [10] established the feasibility of virtual context management, demonstrating that the operating system analogy—treating the LLM context window as RAM and external storage as disk—provides a principled and effective framework for overcoming the finite context window constraint. By equipping the LLM with function calls to autonomously manage its own memory hierarchy, MemGPT created the illusion of an unbounded context for both conversational agents and document analysis tasks. Structurally, MemGPT represents **1D memory**: memories are discrete text chunks organized in a linear hierarchy of storage tiers.

A-MEM [16] shifted the focus from capacity to quality, introducing a dynamic, self-evolving knowledge network inspired by the Zettelkasten method. By constructing rich multi-attribute notes, autonomously generating semantic links, and continuously evolving the representations of existing memories, A-MEM achieved a “better-for-less” outcome: superior performance on long-term conversational benchmarks at a fraction of the token cost of MemGPT. Structurally, A-MEM represents **2D memory**: notes are nodes in a planar graph connected by dynamically generated semantic edges.

GAM [14] addressed the residual limitations of continuous stream-based memory by physically decoupling rapid episodic buffering from stable semantic consolidation. Its hierarchical graph architecture—a global Topic Associative Network layered above local

Event Progression Graphs with cross-layer links—enables the agent to simultaneously track immediate context and maintain long-term consistency, while its LLM-driven semantic boundary detection ensures that global memory is updated only at semantically complete episode boundaries. Structurally, GAM represents **3D memory**: a multi-layer hierarchy with inter-layer cross-links enabling navigation between different levels of semantic granularity.

The three systems together chart a clear and coherent progression, summarized in Table 6.1:

Table 6.1: *Comparative summary of the three surveyed agent memory systems.*

System	Structure	Key Innovation	Core Mechanism
MemGPT	1D (linear tiers)	Virtual context management	OS-inspired paging, FIFO queue
A-MEM	2D (planar graph)	Dynamic knowledge graph	Zettelkasten notes, memory evolution
GAM	3D (hierarchical)	Decoupled encoding/consolidation	State-based consolidation, guided retrieval

This progression reflects a broader trend in agent memory research: moving from simple data retention toward intelligent, structured knowledge management that supports robust long-term reasoning and maintains consistency across arbitrarily long interaction histories.

6.2 Future Work: Toward Multimodal Agent Memory

Despite their significant advances, all three systems surveyed in this report share a fundamental limitation: they operate exclusively on text modality. The current instantiation of each system encodes agent experiences as textual representations—dialogue turns, summaries, and keyword lists—and performs all retrieval and reasoning over these text-based memory nodes.

This unimodal design limits the agent’s ability to leverage the rich non-verbal information that pervades real-world interactions. Human memory is inherently multimodal: we recall not only what was said, but also what we saw, heard, and physically sensed. A conversational agent interacting with a user over video call has access to facial expressions, gestures, and environmental context that carry significant semantic content. A research agent browsing scientific papers encounters charts, diagrams, and mathematical figures. An embodied robot must integrate visual scene understanding with linguistic instructions. All of these modalities are currently beyond the reach of text-only memory systems.

As noted in the GAM paper [14], a natural and important direction for future work is the extension of hierarchical graph memory to a **native multimodal memory framework**. This would require several key advances:

- **Multimodal memory nodes.** The text-only memory nodes of current systems would need to be replaced by multimodal nodes that combine symbolic textual summaries with modality-specific feature representations—for example, keyframe embeddings for visual content and spectral or acoustic embeddings for audio signals. The memory encoding pipeline would need to process raw multimodal inputs and extract modality-appropriate representations as part of note construction.
- **Cross-modal link generation.** The link generation mechanisms of 2D and 3D memory systems would need to be extended to identify meaningful connections not only within but also across modalities. A visual memory of a whiteboard diagram should be linkable to the textual memories of the surrounding discussion, and an audio memory of a specific speaker’s voice should be connectable to the linguistic content of their utterances.
- **Multimodal semantic boundary detection.** GAM’s state-switching mechanism currently detects topic boundaries from lexical topic changes. A multimodal extension would need to detect semantic shifts from visual cues (e.g., a scene change in a video), acoustic signals (e.g., a shift in speaking style or emotional tone), or the combination of multiple modalities—enabling more robust and context-sensitive consolidation triggers.
- **Grounded multimodal retrieval.** Retrieval queries in a multimodal system may themselves be multimodal: an agent might need to find past interactions associated with a specific face, object, or sound. The retrieval mechanism would need to support cross-modal queries, matching a visual query against stored textual memories and vice versa.

Building native multimodal agent memory represents a significant step toward agents that interact with the world in the full richness of human experience—not just reading and writing text, but seeing, hearing, and reasoning over the complete sensory fabric of their interactions. This remains an open frontier for future research.

References

- [1] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav, “Mem0: Building production-ready ai agents with scalable long-term memory,” arXiv, 2025, preprint. [Online]. Available: <https://arxiv.org/abs/2504.19413>
- [2] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, “From local to global: A graph rag approach to query-focused summarization,” arXiv, 2024, preprint. [Online]. Available: <https://arxiv.org/abs/2404.16130>
- [3] Z. Guo, L. Xia, Y. Yu, T. Ao, and C. Huang, “Lightrag: Simple and fast retrieval-augmented generation,” arXiv, 2024, preprint. [Online]. Available: <https://arxiv.org/abs/2410.05779>
- [4] Y. Hu, S. Liu, Y. Yue, G. Zhang, B. Liu, F. Zhu, J. Lin, H. Guo, S. Dou, Z. Xi, S. Jin, J. Tan, Y. Yin, J. Liu, Z. Zhang, Z. Sun, Y. Zhu, H. Sun, B. Peng, Z. Cheng, X. Fan, J. Guo, X. Yu, Z. Zhou, Z. Hu, J. Huo, J. Wang, Y. Niu, Y. Wang, Z. Yin, X. Hu, Y. Liao, Q. Li, K. Wang, W. Zhou, Y. Liu, D. Cheng, Q. Zhang, T. Gui, S. Pan, Y. Zhang, P. Torr, Z. Dou, J.-R. Wen, X. Huang, Y.-G. Jiang, and S. Yan, “Memory in the age of ai agents,” 2026. [Online]. Available: <https://arxiv.org/abs/2512.13564>
- [5] J. Kang, M. Ji, Z. Zhao, and T. Bai, “Memoryos: Memory os for ai agent,” arXiv, 2025, preprint. [Online]. Available: <https://arxiv.org/abs/2506.06326>
- [6] K.-H. Lee, X. Chen, H. Furuta, J. Canny, and I. Fischer, “Readagent: A system for long-context question answering with virtual memory paging,” arXiv, 2024, preprint. [Online]. Available: <https://arxiv.org/abs/2402.09727>
- [7] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” arXiv, 2020, preprint. [Online]. Available: <https://arxiv.org/abs/2005.11401>
- [8] N. Luhmann, “Communicating with slip boxes,” in *Öffentliche Meinung und sozialer Wandel*, H. Baier, P. Kevenhörster, and H. Niklas, Eds. Opladen: Westdeutscher Verlag, 1981, pp. 222–228, original title: Kommunikation mit Zettelkästen. Translated by Manfred Kuehn.
- [9] A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, and Y. Fang, “Evaluating very long-term conversational memory of llm agents,” arXiv, 2024, preprint. [Online]. Available: <https://arxiv.org/abs/2402.17753>

- [10] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "Memgpt: Towards llms as operating systems," arXiv, 2023, preprint. [Online]. Available: <https://arxiv.org/abs/2310.08560>
- [11] J. S. Park, J. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023, pp. 1–22.
- [12] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," arXiv, 2017, preprint. [Online]. Available: <https://arxiv.org/abs/1706.03762>
- [13] B. Wang, X. Liang, J. Yang, H. Huang, S. Wu, P. Wu, L. Lu, Z. Ma, and Z. Li, "Scm: Enhancing large language model with self-controlled memory framework," arXiv, 2023, preprint. [Online]. Available: <https://arxiv.org/abs/2304.13343>
- [14] Z. Wu, H. Zhang, F. Lin, W. Xu, X. Xu, Y. Chen, H. P. Zou, S. Chen, W. Zhang, X. Liu, P. S. Yu, and H. Wang, "Gam: Hierarchical graph-based agentic memory for llm agents," arXiv, 2026, preprint. [Online]. Available: <https://arxiv.org/abs/2604.12285>
- [15] J. Xu, A. Szlam, and J. Weston, "Beyond goldfish memory: Long-term open-domain conversation," arXiv, 2021, preprint. [Online]. Available: <https://arxiv.org/abs/2107.07567>
- [16] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang, "A-mem: Agentic memory for llm agents," arXiv, 2025, preprint. [Online]. Available: <https://arxiv.org/abs/2502.12110>
- [17] W. Zhong, L. Guo, Q. Gao, H. Ye, and Y. Wang, "Memorybank: Enhancing large language models with long-term memory," arXiv, 2023, preprint. [Online]. Available: <https://arxiv.org/abs/2305.10250>